

PLAN DE MEJORA Y MIGRACIÓN

NovaTareas Pro

Ruta integral de desarrollo: correcciones, pruebas,
PostgreSQL, staging en Netcup y demo universitaria cerrada.

Incluido

Código, seguridad, pruebas, PostgreSQL, staging y
preparación operativa.

Regla central

El VPS se usa primero como staging y luego como
demo final cerrada.

Preparado a partir de una revisión del repositorio actual

Julio de 2026

OBJETIVO DEL TRABAJO

1. Resumen ejecutivo

NovaTareas Pro es un proyecto en desarrollo con una base funcional valiosa: dashboard web, API, bot de Telegram, recomendaciones de IA, historial y calendario. Los hallazgos de esta revisión no describen un incidente de producción; son bloqueadores que deben resolverse antes de abrir una beta.

La prioridad no debe ser cambiar de motor de base de datos inmediatamente. Primero hay que evitar pérdida de información, cerrar vulnerabilidades y conseguir un esquema reproducible. Después se puede migrar con datos verificables y una ruta de reversión.

Decisión recomendada: corregir y probar por etapas. El servidor Netcup puede utilizarse temprano como staging restringido; la demo final cerrada solo avanza con evidencia.

Resultado esperado

Una demo universitaria comprobable sobre PostgreSQL, validada en Netcup con usuarios y datos ficticios.

Resultado que se evita

Usar el VPS como laboratorio abierto, mezclar staging con la demo final o dejar servicios olvidados expuestos.

Principios que guían el plan

- No destruir datos. Ninguna migración normal debe contener limpiezas masivas.
- Seguridad antes de expansión. Las rutas que cambian estado deben validar identidad, ownership e inputs.
- Evidencia antes de afirmar éxito. Los conteos, relaciones y flujos deben verificarse con pruebas.
- Reversibilidad. El SQLite original se conserva durante toda la migración.
- Una sola fuente de verdad. Esquema, migraciones y lógica compartida deben quedar centralizados.

PROYECTO EN DESARROLLO

2. Estado actual y estrategia de ambientes

El proyecto todavía está en construcción. Por eso conviene aprovechar el servidor para pruebas compartidas, pero sin tratar cada cambio como si ya fuera una versión pública estable.

Ambientes propuestos

Ambiente	Propósito	Datos	Acceso
Local	Desarrollo rápido y pruebas unitarias.	Ficticios y descartables.	Solo desarrolladores.
CI	Tests, lint, build y migraciones automáticas.	Generados por test.	GitHub Actions.
Staging Netcup	Pruebas colaborativas y de navegador en condiciones reales.	Ficticios o anonimizados.	Restringido al equipo.
Demo final	Evaluación universitaria en condiciones similares a producción.	Ficticios y preservables.	Equipo, docentes e invitados.

Reglas para empezar a probar en el VPS

- El subdominio de staging debe estar protegido por autenticación adicional o una lista de acceso.
- Staging y demo final deben usar bases, secretos y volúmenes diferentes.
- No copiar datos personales reales a staging.
- No ejecutar migraciones destructivas desde el arranque del contenedor.
- Cada versión desplegada debe corresponder a un commit identificable.
- Los cambios se validan en staging antes de promoverse a la demo final.

Uso correcto del servidor: sí para probar pronto; no como una única instalación donde desarrollo y demo final compartan la misma base.

QUÉ DEBE RESOLVERSE

3. Diagnóstico y prioridades

Prioridad	Hallazgo	Impacto	Respuesta
Crítica	La recuperación puede eliminar cuentas tras intentos fallidos.	Pérdida provocable por terceros.	Eliminar el borrado y rediseñar recuperación.
Crítica	La migración 002 borra usuarios, tareas y subtareas.	Pérdida total de información.	Retirlarla del flujo y crear migraciones incrementales.
Crítica	No existe un esquema inicial completo.	Un clon limpio no puede arrancar.	Crear migración inicial reproducible.
Alta	Valores del historial llegan a innerHTML sin escape.	XSS almacenado en cuentas autenticadas.	Render seguro y eliminación de handlers inline.
Alta	Cron y webhook pueden quedar insuficientemente protegidos.	Ejecuciones o eventos falsificados.	Secretos obligatorios y verificación de origen.
Alta	Fechas DATE se comparan como timestamps numéricos.	Recordatorios próximos no confiables.	Modelo temporal explícito y pruebas de zona horaria.
Media	IA duplicada entre cuatro canales.	Comportamientos y modelos inconsistentes.	Unificar servicio de IA y configuración.
Media	Dashboard monolítico y cobertura parcial.	Cambios difíciles de probar y mantener.	Separación gradual y pruebas de flujos.

Advertencia operativa: no ejecutar el comando actual `npm run migrate` sobre una base con información. Antes debe reemplazarse el flujo destructivo.

PRIMER BLOQUE DE EJECUCIÓN

4. Etapas 0 y 1: preservar y asegurar

0

Inventario y copia de seguridad

Duración estimada: 1 a 2 días

- Confirmar si existe una base SQLite real y localizar todas sus copias.
- Crear un respaldo verificable antes de tocar migraciones o esquema.
- Inventariar tablas, columnas, índices, claves foráneas y conteos.
- Identificar datos huérfanos, correos duplicados y fechas inválidas.
- Guardar una copia de solo lectura con hash para demostrar que no fue alterada.

Entregable: informe de inventario y respaldo recuperable.

Criterio de salida: se conoce qué datos existen y se puede volver al punto inicial.

1

Correcciones críticas de seguridad

Duración estimada: 3 a 5 días

Área	Cambio requerido	Prueba mínima
Recuperación	Nunca borrar cuentas; token con crypto; rate limiting; respuestas con hash o recuperación por correo.	Diez fallos no eliminan ni modifican la cuenta.
Sesión	Cookies Secure, HttpOnly y SameSite; rotación al cambiar contraseña.	Cookie segura y token previo invalidado.
XSS	Eliminar innerHTML con datos; usar textContent o componentes; retirar onclick contruidos.	Payloads con HTML y backticks se muestran como texto.
Cron	CRON_SECRET obligatorio y comparación segura.	Sin secreto siempre responde 401.
Telegram	Verificar secreto del webhook y escapar HTML enviado al bot.	Actualización falsificada es rechazada.
Google	OAuth con state, manejo de errores y revocación.	Callback sin state válido es rechazado.

BASE PARA POSTGRESQL

5. Esquema y migraciones confiables

2

Diseñar una fuente de verdad para los datos

Duración estimada: 4 a 6 días

La meta es que una base vacía pueda construirse con un solo comando y que repetir el proceso no borre datos. Se recomienda Drizzle ORM y Drizzle Kit por su cercanía a SQL y su bajo peso.

Cambios de esquema recomendados

Concepto	Decisión propuesta
Identificadores	BIGINT generado o UUID, con una política única para todo el sistema.
Correo	Normalizado, NOT NULL y UNIQUE; considerar extensión citext en PostgreSQL.
Estado	CHECK o enum controlado: pendiente, en_progreso, completada.
Prioridad	CHECK o enum controlado: baja, media, alta, urgente.
Fecha límite	DATE cuando solo importa el día.
Recordatorio	TIMESTAMPTZ cuando debe dispararse en un instante específico.
Auditoría	created_at y updated_at con zona horaria; historial con actor y campo cambiado.
IA	Tabla de recomendaciones separada de las subtareas reales.
Google	Columnas de tokens definidas explícitamente y valores cifrados.

Entregables

- Migración inicial completa.
- Migraciones incrementales sin DELETE masivos.
- Diagrama entidad-relación y diccionario de datos.
- Comando de creación para desarrollo y pruebas.
- Prueba automatizada que crea la base desde cero.

ANTES DE MOVER LOS DATOS

6. Pruebas y estabilización funcional

3

Convertir los riesgos en pruebas repetibles

Duración estimada: 4 a 6 días

Las pruebas existentes cubren principalmente la API inteligente. Antes de migrar deben cubrirse las operaciones que protegen usuarios y datos.

Bloque	Escenarios obligatorios
Autenticación	Registro, login, logout, expiración, cambio y recuperación de contraseña.
Autorización	Un usuario no puede leer, editar, comentar, archivar ni borrar tareas ajenas.
Tareas	Crear, editar, completar, reabrir, archivar, desarchivar y eliminar.
Historial	Registrar cambios correctos sin ejecutar HTML almacenado.
Fechas	Vencidas, próximas, zona America/El_Salvador y límites de día.
Integraciones	Cron y Telegram con secretos; Google OAuth con state.
IA	z.ai disponible, Ollama disponible y fallback sin red.
Migraciones	Base vacía, base antigua y reejecución sin pérdida.

Verificación automática

Tests unitarios y de integración, lint, build y migraciones en CI sin claves externas reales.

Verificación de interfaz

Navegador real en escritorio y móvil: autenticación, tareas, modales, calendario, foco y payloads maliciosos.

Puerta de avance: no iniciar la migración a PostgreSQL hasta que los flujos críticos pasen de forma determinista sobre el esquema estabilizado.

CAMBIO DE MOTOR

7. Migración de SQLite a PostgreSQL

4

Migrar sin perder trazabilidad

Duración estimada: 5 a 8 días

Estrategia recomendada

Paso	Acción	Resultado
1	Preparar	Crear PostgreSQL de pruebas, aplicar migraciones nuevas y congelar el esquema durante el ensayo.
2	Extraer	Leer SQLite en modo solo lectura y exportar cada tabla con identificadores originales.
3	Transformar	Normalizar fechas, booleanos, correos y valores de estado; documentar cada corrección.
4	Cargar	Importar en orden de dependencias dentro de transacciones.
5	Validar	Comparar conteos, IDs, relaciones, fechas y una muestra funcional por usuario.
6	Probar	Ejecutar toda la suite y los flujos manuales contra PostgreSQL.
7	Conservar	Mantener SQLite intacto como respaldo hasta la aceptación final.

Consultas que requieren adaptación

- GROUP_CONCAT y serialización artesanal de subtareas.
- datetime('now'), unixepoch() y conversiones numéricas de fechas.
- Booleanos almacenados como 0 y 1.
- Últimos IDs insertados y tipos de retorno.
- Manejo de concurrencia y transacciones.

No se recomienda doble escritura SQLite/PostgreSQL. Para este tamaño de proyecto es más seguro ensayar, establecer una ventana breve sin escrituras, importar, validar y cambiar.

DESPUÉS DE POSTGRESQL

8. Consolidación y mantenibilidad

5

Reducir duplicación y dejar el proyecto transferible

Duración estimada: 4 a 6 días

Servicio único de IA

Centralizar proveedor, modelo, timeouts, prompts, fallbacks, rate limiting y registro de uso.

Capa única de datos

Evitar SQL disperso en páginas, bot y utilidades; usar repositorios compartidos.

Dashboard modular

Separar render, estado, API y componentes sin intentar una reescritura total.

Procesos claros

Distinguir aplicación web, bot y recordatorios; documentar cómo se inician y se detienen.

Documentación que debe quedar

- README corregido con instalación reproducible.
- Variables de entorno completas, solo con placeholders.
- Mapa del esquema y reglas de negocio.
- Comandos de pruebas y migración.
- Procedimiento de exportación y restauración.
- Limitaciones conocidas y funciones todavía pendientes.

Qué se considera terminado

- El repositorio puede instalarse desde cero sin archivos ocultos ni una base heredada.
- Los usuarios existentes pueden iniciar sesión después de la migración.
- Los conteos y relaciones coinciden entre SQLite y PostgreSQL.
- Los flujos críticos pasan en CI y en navegador.
- No quedan rutas conocidas capaces de borrar cuentas por intentos externos.

COMPLETAR EL PRODUCTO

9. Funcionalidades e integraciones pendientes

6

Cerrar flujos antes de la demo final

Duración estimada: 4 a 7 días

Componente	Trabajo requerido	Criterio de aceptación
Telegram	Persistir sesiones o manejar reinicios; corregir errores; validar ownership de categorías.	Crear, listar y recomendar sin perder el flujo tras errores.
Recordatorios	Separar fecha límite de instante de aviso; reprogramar al cambiar la fecha.	Casos próximos y vencidos se envían una sola vez.
Google Calendar	Migraciones de tokens, state, cifrado, revocación y estados de conexión.	Conectar, consultar y desconectar sin exponer tokens.
IA y RAG	Unificar modelos, indexación, fuente reportada, límites y registro de fallbacks.	Todos los canales usan la misma configuración.
API inteligente	Mantenerla interna o protegerla con una clave para la demostración; aplicar cuota y tamaño máximo.	No puede consumir saldo sin límite ni saltarse el contrato.
Avatares	Validar contenido real, limpiar archivos anteriores y abstraer almacenamiento.	Solo imágenes válidas y sin crecimiento indefinido.

Trabajo de interfaz y experiencia

- Dividir el dashboard de casi 3,000 líneas en componentes y módulos gradualmente.
- Revisar navegación por teclado, foco, modales, mensajes de error y contraste.
- Probar anchos móviles y escritorio con tareas largas, sin fecha y muchos comentarios.
- Eliminar logs de depuración visibles y estados silenciosos cuando una petición falla.
- Evitar cambios visuales especulativos hasta que los flujos básicos sean claros.

10. Staging en el servidor Netcup

7

Usar el VPS como entorno controlado

Puede comenzar después de cerrar los riesgos críticos

No hace falta esperar a que todo el proyecto esté terminado para probarlo en Netcup. Después de la etapa de seguridad mínima puede publicarse un staging restringido con datos ficticios.

Arquitectura mínima del staging

Elemento	Decisión
Entrada	Subdominio separado, HTTPS y acceso restringido al equipo.
Aplicación	Contenedor Astro/Node identificado por commit y sin edición manual dentro del servidor.
Base	PostgreSQL de staging independiente y sin puerto 5432 público.
Procesos	Web, bot y scheduler separados, con reinicio y health checks.
Secretos	Archivo o gestor fuera de Git; credenciales distintas a la demo final.
Datos	Semillas ficticias reproducibles; nunca una copia sin anonimizar.

Ciclo de prueba en staging

- Subir una versión candidata desde Git.
- Aplicar migraciones primero sobre una base de staging respaldada.
- Ejecutar smoke tests automáticos.
- Realizar pruebas de navegador con el equipo.
- Registrar errores y reproducirlos localmente.
- Corregir, volver a desplegar y conservar el historial de versiones.

Condición: mientras sea staging no se invita a usuarios externos ni se promete disponibilidad. Los datos pueden reiniciarse de forma planificada.

PROMOCIÓN Y CIERRE

11. De staging a demo final cerrada

8

Habilitar la demostración solo después de aceptación

Duración estimada: 2 a 4 días

Puertas obligatorias antes de la demostración

- Suite completa, lint y build aprobados desde un commit etiquetado.
- Restauración de PostgreSQL ensayada, no solamente un archivo de respaldo existente.
- Recuperación de contraseña, cron, Telegram, XSS y ownership verificados.
- Secretos de la demo separados y pertenecientes a los responsables del proyecto.
- Registro cerrado, usuarios ficticios preparados y acceso limitado a participantes.
- Rollback documentado y versión anterior disponible.

Mismo VPS, ambientes separados

Si staging y demo comparten Netcup, deben conservar bases, volúmenes, secretos y subdominios independientes.

Servicio temporal

Definir fecha de cierre, archivo final, revocación de accesos y apagado para no dejar servicios olvidados.

Operación mínima durante el servicio

Frecuencia	Actividad
Cada despliegue	Backup previo, migración, smoke test y observación posterior.
Antes de cambios	pg_dump y respaldo de configuración antes de migraciones o actualizaciones.
Semanal	Revisar espacio, errores, respaldos y dependencias durante el proyecto.
Antes de exponer	Probar restauración y revisar accesos de la demo.
Al finalizar	Archivar código y evidencia, exportar la base ficticia si interesa y apagar servicios.

Netcup indica que sus copias voluntarias de web hosting no tienen disponibilidad garantizada. Por eso este plan exige respaldos propios y una copia externa independiente.

PLAN RECOMENDADO

12. Cronograma y puertas de decisión

Semana	Trabajo principal	Puerta de salida
1	Inventario, respaldo y seguridad crítica.	Base preservada y bloqueadores críticos cerrados.
2	Esquema inicial, migraciones seguras y staging restringido.	Instalación reproducible y servidor de prueba controlado.
3	Pruebas de autenticación, ownership, tareas y fechas.	Flujos críticos estables en local y CI.
4-5	Adaptación de datos y ensayos SQLite a PostgreSQL.	Importación completa con conteos válidos.
6	Consolidación de arquitectura, IA y dashboard.	Menor duplicación y fallos reproducibles.
7	Telegram, Google, recordatorios y QA en staging.	Integraciones demostrables con datos ficticios.
8	Demo final, documentación, evidencia y cierre.	Evaluación universitaria lista y servicio cerrable.

La estimación integral es de 6 a 8 semanas. Al trabajar solo con usuarios ficticios se simplifica la migración y la privacidad, pero no conviene recortar seguridad, pruebas ni restauración.

Dependencias entre etapas

No avanzar a...	Hasta comprobar...
Pruebas integrales	Que la base pueda crearse de forma reproducible.
PostgreSQL	Que el esquema y los flujos críticos estén estabilizados.
Demo cerrada	Que los datos coincidan y exista una restauración ensayada.
Cierre del servidor	Que código, evidencias y respaldos necesarios estén archivados.

CIERRE DE PLANIFICACIÓN

13. Aceptación, alcance y próximos pasos

Criterios de aceptación globales

- No existe ninguna migración normal que elimine datos de usuarios.
- Una instalación limpia crea todas las tablas y columnas necesarias.
- Los controles de autenticación y ownership tienen pruebas negativas.
- Los recordatorios utilizan tipos de fecha coherentes y casos de zona horaria.
- El HTML proveniente de usuarios o modelos se presenta de forma segura.
- La migración SQLite a PostgreSQL produce conteos y relaciones equivalentes.
- El SQLite original permanece disponible como respaldo durante la aceptación.
- La documentación distingue claramente lo terminado de lo pendiente.

Límites de esta versión universitaria

El objetivo es una demostración cerrada, no un producto comercial abierto. No se plantea alta disponibilidad, soporte 24/7, escalamiento horizontal, cobros ni tratamiento de datos personales reales. Si el proyecto continúa después de la universidad, esos temas requieren una revisión nueva.

Siguiente decisión

Confirmar la fecha de la demostración y quiénes tendrán acceso al staging y a la demo final.

Primera acción segura

Hacer inventario y respaldo. No ejecutar todavía el comando actual de migraciones.

Evidencia principal revisada

- package.json y migraciones.
- src/pages/api/auth/recover.js.
- src/lib/db.js y rutas de tareas.
- migrations/002_new_user_schema.cjs.
- src/pages/dashboard.astro.
- aiEngine.js, rag.js y Telegram.

Este plan es una guía de ejecución técnica basada en el estado actual del repositorio. Cada etapa debe cerrarse con evidencia antes de iniciar la siguiente.